MLFF PERF-P2 Lazy TARGET-DATA2C Ladder Specification

Decision-equivalent early termination with explicit v2 authority

mdstats project

2026-08-15

Contents

Status

Gate: `PERF-P2`
Release: `mdstats 0.20.181a0`
Authority class: A
Implementation status: complete for bounded deterministic and supplied-data-derived qualification
Current status: historical and superseded for generated campaigns by `SIZE-HALVE1` (`0.20.182a0`)

> **Supersession notice.** This document remains the accurate historical specification of `0.20.181a0`. Its generated-campaign scientific premise is no longer current. `SIZE-HALVE1` makes coverage a hard admissibility gate and requires every coverage-qualified size to enter a 3-epoch learning screen. Current generated campaigns therefore require TARGET-DATA2C v3 full-ladder authority. See `mlff_size_halve1_target_size_revision_spec.md`.

PERF-P2 replaces exhaustive TARGET-DATA2C ladder materialization with a lazy, fail-closed v2 authority. It may omit larger configured rungs only after the smallest Stage-A shortlist is already fixed by the same frozen predicates used by TARGET-DATA2D.

Scientific contract

For configured rung sizes

$$K_1 < K_2 < \cdots < K_m,$$

TARGET-DATA2C preserves one exact nested selector prefix. For label domain d, define

$$Q_d(K) = C_d(K) \wedge M_d(K),$$

where $C_d(K)$ is the frozen TARGET-DATA2B coverage/extent/stratum predicate and $M_d(K)$ is the mandatory-obligation predicate. The global Stage-A predicate is

$$Q(K) = \bigwedge_d Q_d(K).$$

PERF-P2 relies only on the declared nested monotonicity contract: over the materialized exact prefix, a previously satisfied required coverage, extent, stratum, mandatory-obligation, or aggregate Stage-A predicate may not reverse at a larger rung. Every observed transition is audited at runtime.

Let s be `TargetSizeConvergencePolicy.max_short_training_candidates`. The canonical campaign has $s = 4$. Lazy materialization may terminate at the first rung K_j for which exactly the s smallest observed qualifying rungs are known. The implementation does **not** hard-code four; the stop width is bound to the active TARGET-DATA2D policy.

TARGET-DATA2C v2 authority

`TargetDataLadderPlan.v2` records:

- the complete configured candidate-size sequence;
- the exact materialized candidate-size prefix;
- one global qualification record for every materialized rung;
- `stage_a_survivor_limit`;
- the qualifying sizes that establish an early stop;
- intentionally unmaterialized larger sizes;
- sizes unavailable because a label-domain pool is too small;
- the monotonicity contract version;
- the last materialized rung;
- the materialization stop reason; and
- the existing per-domain exact membership, ordering, coverage, and mandatory-obligation evidence for each materialized rung.

Materialized, intentionally unmaterialized, and pool-unavailable sizes must form an exact non-overlapping partition of the configured sequence. The materialized sequence must be an exact configured prefix.

An intentionally unmaterialized rung is **not** a failed rung. It is absent from domain rung records and represented by explicit plan-level evidence. TARGET-DATA2D consumes only the materialized v2 qualification records.

Legacy compatibility and migration

Legacy `TargetDataLadderPlan.v1` remains readable and digest-stable for regression qualification. It is not valid current campaign authority after PERF-P2. Campaign restart validation rejects v1 as stale and rebuilds v2 from the authenticated TARGET-DATA2B reference and role freeze.

Schema and authority-version pairing is fail-closed. A v2 schema carrying a non-v2 authority version, or a legacy schema carrying a conflicting version, is rejected during deserialization.

Exact execution path

PERF-P2 reuses the PERF-P1 `ExactFPSState`. For each configured rung it:

1. extends the exact nested selector only to the next rung;
2. scores that rung exactly;
3. records per-domain rung evidence;
4. forms the global Stage-A-equivalent qualification record;
5. audits monotonicity against the previous materialized rung; and
6. stops only when `stage_a_survivor_limit` global qualifiers have been established.

If the stop condition is never reached, v2 materializes every globally available configured rung. Existing minimum-feasibility rules are unchanged.

`coverage_query_workers` is execution-only. Exact `cKDTree.query` calls retain `eps=0`; worker-count changes must leave the v2 content digest unchanged [2]. The underlying farthest-first traversal remains the exact deterministic PERF-P1 implementation; PERF-P2 changes only when that existing traversal is allowed to stop [1].

Qualification contract

For every deterministic qualification corpus, exhaustive v1 and lazy v2 must produce:

- identical Stage-A survivor sizes;
- identical selected-frame membership and prefix ordering for every survivor;
- identical survivor coverage-report digests and pass/fail status;
- identical mandatory-obligation status for every survivor; and
- the same TARGET-DATA2D Stage-A shortlist.

Additional gate tests require:

- exhaustive fallback when the shortlist cannot be fixed early;
- global stopping across multiple label domains;
- configurable shortlist widths;
- v1 read compatibility and stale-current-authority rejection;
- v2 serialization/digest round trips;
- worker-count invariance; and
- fail-closed rejection of any observed monotonicity reversal.

The v2 plan digest is expected to differ from v1 because the authority schema and intentional materialization boundary differ.

CPU qualification

The benchmark uses the complete PERF-P0 native coverage reference with 37,633 target frames. Frame indices are deterministically remapped to the DATA2C role-order contract. Two fixtures are evaluated.

Exhaustive fallback fixture

The original coverage radii and extents are retained. Only 2048, 4096, and 8192 qualify, so v2 must reach the configured maximum. v1 and v2 produce the same Stage-A survivors `(2048, 4096, 8192)` and identical survivor evidence. Fresh-process timing ranges overlap and are scheduler-sensitive; no fallback speedup is claimed.

Forced early-stop fixture

The same numerical reference arrays are used, but local radii are made intentionally permissive and extent channels are removed solely to force the monotone early-stop branch. This is an execution-qualification fixture, not a new production coverage policy.

Three fresh-process samples on the available Intel Xeon Platinum 8573C host under an 8-core cgroup quota and 4 GiB memory limit give:

Metric	Exhaustive v1	Lazy v2	Change
Median wall	7.867 s	1.556 s	**-80.23%**
Wall range	7.602–8.898 s	1.505–1.713 s	non-overlapping
Median peak RSS	327.07 MiB	316.23 MiB	−3.31%
Materialized rungs	7	4	−3 rungs
Master-order entries	8192	1024	8x fewer
Serialized authority	4,729,481 B	591,058 B	**-87.50%**

The v1 and v2 Stage-A survivor sizes are both `(128, 256, 512, 1024)`, and the survivor scientific signature is identical:

`1b91c9790753ccef5de367609ed4a452af6632c5d112a4f84bc922b329a4c261`.

Workers 1 and 4 produce the same v2 plan digest:

`af29ca65e44741e7e5b49b5da16cbe1a37b8b761b9e3f51be81641e6f75f7db9`.

The benchmark scientific digest is:

`ae55c560995791174ac63e2d894ec685d74a02c389eb0a955d87e77cfd9f18f9`.

Acceptance decision

PERF-P2 passes bounded qualification because the new authority changes only which scientifically irrelevant larger rungs are materialized after Stage A is already fixed. Survivor membership, survivor reports, mandatory status, and the downstream Stage-A shortlist remain exact against the exhaustive v1 oracle.

The measured performance claim is intentionally narrow: large gains occur when the early-stop condition is reached. PERF-P2 does not claim that independent one-rung coverage scoring is intrinsically faster than PERF-P1 progressive scoring on an exhaustive ladder.

Evidence

- `benchmarks/benchmark_mlff_perf_p2.py`
- `audits/analysis/mlff_perf_p2_lta_cloud_cpu_2026-08-15.json`
- `release/MLFF_PERF_P2_QUALIFICATION_0.20.181a0.json`
- `tests/test_mlff_perf_p2.py`

No authorizing MACE-MH-1 checkpoint or GPU runtime was supplied. PERF-P2 makes no TRAIN2/EVAL2, GPU-memory, GPU-throughput, OOM, or production training-time claim.

References

[1] T. F. Gonzalez, "Clustering to Minimize the Maximum Intercluster Distance," *Theoretical Computer Science* **38**, 293–306 (1985). DOI: 10.1016/0304-3975(85)90224-5.

[2] SciPy developers, "scipy.spatial.cKDTree.query," SciPy reference documentation. Available at: https://docs.scipy.org/doc/scipy/reference/generated/scipy.spatial.cKDTree.query.html (accessed 2026-08-15).